Using Android Data and Storage API’s

Data storage in android

- Shared Preferences:-Store private primitive data in key-value pairs.
- Internal Storage:-Store private data on the device memory.
- External Storage:-Store public data on the shared external storage.
- SQLite Databases:-Store structured data in a private database.

Shared Preferences

Android provides simple preferences system for storing primitives application data at the Activity level.

And Preferences shared across all of an application’s activities.

You can not share preferences outside of the package.

Preferences are stored as groups of key value pairs.

Datatypes

The following data types are supported as preference settings:

Boolean values

Float values

Integer values

Long values

String values

Shared Preferences

To add preferences to your application, you must take the following steps:

1. Retrieve an instance of a SharedPreferences object
2. Create a SharedPreference.Editor to modify preference content
3. Make changes to preference using editor
4. Commit your changes.

Creating Private and Shared Preferences

You can create a new shared preference file or access an existing one by calling one of these methods:

- **getSharedPreferences():** Use this if you need multiple shared preference files identified by name, which you specify with the first parameter. You can call this from any Context in your app.

- **getPreferences():** Use this from an Activity if you need to use only one shared preference file for the activity. Because this retrieves a default shared preference file that belongs to the activity, you don't need to supply a name.

  For example, the following code accesses the shared preferences file that's identified by the resource string `R.string.preference_file_key` and opens it using the private mode so the file is accessible by only your app:

Contd:

```
Context context = getActivity();
SharedPreferences sharedPref = context.getSharedPreferences(
        getString(R.string.preference_file_key), Context.MODE_PRIVATE);
```

- When naming your shared preference files, you should use a name that's uniquely identifiable to your app. A good way to do this is prefix the file name with your application ID.

  For example: "com.example.myapp.PREFERENCE_FILE_KEY"

  Alternatively, if you need just one shared preference file for your activity, you can use the getPreferences() method:

  ```
  SharedPreferences sharedPref = getActivity().getPreferences(Context.MODE_PRIVATE);
  ```

Modes

public abstract SharedPreferences getSharedPreferences (String name, int mode)

This method takes two arguments, the first being the name of the **SharedPreference(SP) file** and the other is the **context mode** that we want to store our file in.

MODE_PUBLIC will make the file public which could be accessible by other applications on the device

MODE_PRIVATE keeps the files private and secures the user's data.

MODE_APPEND is used while reading the data from the SP file.

Nested classes of Shared Preferences

1. **SharedPreferences.Editor**: Interface used to write(edit) data in the SP file. Once editing has been done, one must **commit()** or **apply()** the changes made to the file.
2. **SharedPreferences.OnSharedPreferenceChangeListener()**: Called when a shared preference is changed, added, or removed. This may be called even if a preference is set to its existing value. This callback will be run on your main thread.

The methods of Shared Preferences for searching & reading

- **contains(String key)**: This method is used to check whether the preferences contain a preference.
- **edit()**: This method is used to create a new Editor for these preferences, through which you can make modifications to the data in the preferences and atomically commit those changes back to the SharedPreferences object.
- **getAll()**: This method is used to retrieve all values from the preferences.
- **getBoolean(String key, boolean defValue)**: This method is used to retrieve a boolean value from the preferences.
- **getFloat(String key, float defValue)**: This method is used to retrieve a float value from the preferences.
- **getInt(String key, int defValue)**: This method is used to retrieve an int value from the preferences.

The methods for searching Adding, Updating and Deleting

Editor.clear(): Removes all preferences. This operation happens first, regardless of when it is called within an editing session; then all other changes are made and committed.

Editor.remove(): Removes a specific preference by name. This operation happens first, regardless of when it is called within an editing session; then all other changes are made and committed.

Editor.putBoolean(): Sets a specific Boolean-type preference by name.

Editor.putFloat(): Sets a specific Float-type preference by name.

Editor.putInt(): Sets a specific Integer-type preference by name.

Editor.putLong(): Sets a specific Long-type preference by name

Editor.putString(): Sets a specific String-type preference by name

Editor.commit(): Commits all changes from this editing session.

Contd.

- **getLong(String key, long defValue)**: This method is used to retrieve a long value from the preferences.
- **getString(String key, String defValue)**: This method is used to retrieve a String value from the preferences.
- **getStringSet(String key, Set defValues)**: This method is used to retrieve a set of String values from the preferences.

A sample code to write Data in Shared Preferences:

```
// Storing data into SharedPreferences

        SharedPreferences sharedPreferences =
    getSharedPreferences("MySharedPref",MODE_PRIVATE);

// Creating an Editor object to edit(write to the file)

    SharedPreferences.Editor myEdit = sharedPreferences.edit();

// Storing the key and its value as the data fetched from edittext

    myEdit.putString("name", name.getText().toString());

    myEdit.putInt("age", Integer.parseInt(age.getText().toString()));

// Once the changes have been made, we need to commit to apply those changes made,

// otherwise, it will throw an error

    myEdit.commit();
```

The sample code to read Data in Shared Preferences:

```
// Retrieving the value using its keys the file name  must be same in both saving and retrieving the data

        SharedPreferences sh = getSharedPreferences("MySharedPref", MODE_APPEND);

// The value will be default as empty string because for the very first time when the app is opened, there is
// nothing to show

        String s1 = sh.getString("name", "");

        int a = sh.getInt("age", 0);

// We can then use the data

        name.setText(s1);

        age.setText(String.valueOf(a));
```

Internal Storage in Android

Saving and loading data on the internal storage is private for an application

That can not be accessed by other applications.

When the app is uninstalled the data stored in the internal by that app is removed.

To read and write in the android internal storage we have methods

- **OpenFileOutput():** used for creating and saving a file. This method returns a FileOutputStream instance.

  Syntax: OpenFileOutput(String filename,int mode)

  Context.MODE_PRIVATE: If the file exists then it is overridden else a new file is created.

  Context.MODE_APPEND: if the file exists then the data is appended at the end of the file.

  Returns: FileOutputStream object
- **OpenFileInput():** Used to read data from a file, this returns an FileInputStream instance.

  Syntax: OpenFileInput(String filename)

  Returns: FileInputStream object

- **deleteFile():** Deletes an application file by name.
- **fileList():** Gets a list of all files in the file subdirectory.
- **getFilesDir():** Retrieves the application files subdirectory object.
- **getCacheDir():** Retrieves the application/cache subdirectory object
- **getDir():** Creates or retrieves an application subdirectory by name.

Creating and writing to files

Below code creates and opens a file called Filename.txt. We can write a single line of text to the file and then close it.

```
FileOutputStream fos;

String strfileContents = “Some text to write to the file.”;

fos= openFileOutput(“Filename.txt”,”MODE_PRIVATE);

fos.write(strFileContents.getBytes());

fos.close();
```

Contd.

We can append data to file by opening it with the mode set to MODE_APPEND:

```
FileOutputStream fos;

String strfileContents = “More text to write to the file.”;

fos= openFileOutput(“Filename.txt”,”MODE_APPEND);

fos.write(strFileContents.getBytes());

fos.close();
```

Reading from files

Reading from files in default application directory

Following code opens a file called Filename.txt for read operation:

```
String strFileName= “Filename.txt”;
FileInputStream fis = openFileInput(strFileName);
```

Reading Raw files Byte-by-Byte
Example to read a text file,line by line, and store it in a StringBuffer:

```
FileInputStream fis = openFileInput(filename);
StringBuffer sBuffer = new StringBuffer();
DataInputStream dataIO= new DataInputStream(fis);
String strLine = null;
while((strLine = dataIO.readLine()) != null){
 sBuffer.append(strLine + “\n”);
}
dataIO.close();
fis.close();
```

External Storage

Android provides two types of physical storage locations: *internal storage* and *external storage*.

On most devices, internal storage is smaller than external storage.

However, internal storage is always available on all devices, making it a more reliable place to put data on which your app depends.

Removable volumes, such as an SD card, appear in the file system as part of external storage. Android represents these devices using a path, such as /sdcard.

Apps themselves are stored within internal storage by default. If your APK size is very large, however, you can indicate a preference within your app's manifest file to install your app on external storage instead:

<manifest
android:installLocation="preferExternal"
</manifest>

Permissions and access to external storage

Android defines the following storage-related permissions: READ_EXTERNAL_STORAGE, WRITE_EXTERNAL_STORAGE, and MANAGE_EXTERNAL_STORAGE.

On earlier versions of Android, apps needed to declare the READ_EXTERNAL_STORAGE permission to access any file outside the app-specific directories on external storage. Also, apps needed to declare the WRITE_EXTERNAL_STORAGE permission to write to any file outside the app-specific directory.

More recent versions of Android rely more on a file's purpose than its location for determining an app's ability to access, and write to, a given file. In particular, if your app targets Android 11 (API level 30) or higher, the WRITE_EXTERNAL_STORAGE permission doesn't have any effect on your app's access to storage. This purpose-based storage model improves user privacy because apps are given access only to the areas of the device's file system that they actually use.

Android 11 introduces the MANAGE_EXTERNAL_STORAGE permission, which provides write access to files outside the app-specific directory and MediaStore.

External Storage Availability

In order to avoid crashing the app first, we need to check storage SD Card is available for reading and write operations.

The method **getExternalStorageState()** is used to determine the state of mounted storage media such as SD Card is missing, read-only or readable, and writable.

Code to check availability of external storage:

```
boolean isAvailable= false;
boolean isWritable= false;
boolean isReadable= false;
String state = Environment.getExternalStorageState();

if(Environment.MEDIA_MOUNTED.equals(state)) {
// Operation possible - Read and Write
isAvailable= true;
isWritable= true;
isReadable= true;
} else if (Environment.MEDIA_MOUNTED_READ_ONLY.equals(state)) {
        // Operation possible - Read Only
        isAvailable= true;
        isWritable= false;
        isReadable= true;
} else {
        // SD card not available
        isAvailable = false;
        isWritable= false;
        isReadable= false;
}
```

Methods to Store data in External Storage

- **getExternalStoragePublicDirectory():** This is the present recommended method to keep files public and these files are not deleted even when the app is uninstalled from the system. For eg: Images clicked by the camera are still available even after we uninstall the camera.
- **getExternalFilesDir(String type):** This method is used to store private data that are specific to the app only. And data are removed as we uninstall the app.
- **getExternalStorageDirectory():** This method is not recommended. It is now absolute and it is used to access external storage in older versions, API Level less than 7.

Access Permission to External Storage

```
<manifest ... >
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
    <application ... >
        <activity android:name=".MainActivity" ... >
        ...
        </activity>
    </application>
</manifest>
```

MainActivity

```
// After API 23 the permission request for accessing external storage is changed
    // Before API 23 permission request is asked by the user during installation of app
    // After API 23 permission request is asked at runtime
    private int EXTERNAL_STORAGE_PERMISSION_CODE = 23;

public void savePublicly(View view) {
        // Requesting Permission to access External Storage
        ActivityCompat.requestPermissions(this, new String[]{Manifest.permission.READ_EXTERNAL_STORAGE},
                EXTERNAL_STORAGE_PERMISSION_CODE);
        String editTextData = editText.getText().toString();

        // getExternalStoragePublicDirectory() represents root of external storage, we are using DOWNLOADS
        // We can use following directories: MUSIC, PODCASTS, ALARMS, RINGTONES, NOTIFICATIONS, PICTURES,
MOVIES
        File folder = Environment.getExternalStoragePublicDirectory(Environment.DIRECTORY_DOWNLOADS);

        // Storing the data in file with name as geeksData.txt
        File file = new File(folder, "geeksData.txt");
        writeTextData(file, editTextData);
        editText.setText("");
    }
```

Contd.

```
public void savePrivately(View view) {
        String editTextData = editText.getText().toString();

        // Creating folder with name GeeksForGeeks
        File folder = getExternalFilesDir("GeeksForGeeks");

        // Creating file with name gfg.txt
        File file = new File(folder, "gfg.txt");
        writeTextData(file, editTextData);
        editText.setText("");
    }
```

```
// writeTextData() method save the data into the file in byte format
    // It also toast a message "Done/filepath_where_the_file_is_saved"
    private void writeTextData(File file, String data) {
        FileOutputStream fileOutputStream = null;
        try {
            fileOutputStream = new FileOutputStream(file);
            fileOutputStream.write(data.getBytes());
            Toast.makeText(this, "Done" + file.getAbsolutePath(), Toast.LENGTH_SHORT).show();
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            if (fileOutputStream != null) {
                try {
                    fileOutputStream.close();
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }
        }
    }
```

SQLite Database

SQLite Database is an open-source database provided in Android which is used to store data inside the user’s device in the form of a Text file.

We can perform so many operations on this data such as adding new data, updating, reading, and deleting this data.

SQLite is an offline database that is locally stored in the user’s device and we do not have to create any connection to connect to this database.

Contd.

Database Package: The main package is android.database.sqlite that contains the classes to manage your own databases

Database Creation: In order to create a database you just need to call this method openOrCreateDatabase with your database name and mode as a parameter. It returns an instance of SQLite database which you have to receive in your own object.Its syntax is given below:

SQLiteDatabase mydatabase = openOrCreateDatabase("your database name",MODE_PRIVATE,null);

Database Package Functions:

1. **openDatabase(String path, SQLiteDatabase.CursorFactory factory, int flags, DatabaseErrorHandler errorHandler)**

This method only opens the existing database with the appropriate flag mode. The common flags mode could be OPEN_READWRITE OPEN_READONLY

2. **openDatabase(String path, SQLiteDatabase.CursorFactory factory, int flags)**

It is similar to the above method as it also opens the existing database but it does not define any handler to handle the errors of databases

3. **openOrCreateDatabase(String path, SQLiteDatabase.CursorFactory factory)**

It not only opens but create the database if it not exists. This method is equivalent to openDatabase method.

4. **openOrCreateDatabase(File file, SQLiteDatabase.CursorFactory factory)**

This method is similar to above method but it takes the File object as a path rather then a string. It is equivalent to file.getPath()

Database - Insertion

we can create table or insert data into table using execSQL method defined in SQLiteDatabase class. Its syntax is given below

```
mydatabase.execSQL("CREATE TABLE IF NOT EXISTS Student(Username VARCHAR,Password VARCHAR);");
  mydatabase.execSQL("INSERT INTO Student VALUES('admin','admin');");
```

execSQL(String sql, Object[] bindArgs)
This method not only insert data , but also used to update or modify already existing data in database using bind arguments

Database - Fetching

We can retrieve anything from database using an object of the Cursor class. We will call a method of this class called rawQuery and it will return a resultset with the cursor pointing to the table. We can move the cursor forward and retrieve the data.

```
Cursor resultSet = mydatabase.rawQuery("Select * from Student",null);
resultSet.moveToFirst();
String username = resultSet.getString(0);
String password = resultSet.getString(1);
```

Methods of Database Fetching

1. **getColumnCount():** This method return the total number of columns of the table.
2. **getColumnIndex(String columnName):** This method returns the index number of a column by specifying the name of the column
3. **getColumnName(int columnIndex):** This method returns the name of the column by specifying the index of the column
4. **getColumnNames():** This method returns the array of all the column names of the table.
5. **getCount():** This method returns the total number of rows in the cursor
6. **getPosition():** This method returns the current position of the cursor in the table
7. **isClosed():** This method returns true if the cursor is closed and return false otherwise

Database - Helper class

For managing all the operations related to the database , an helper class has been given and is called SQLiteOpenHelper. It automatically manages the creation and update of the database. Its syntax is given below

```
public class DBHelper extends SQLiteOpenHelper {
   public DBHelper(){
      super(context,DATABASE_NAME,null,1);
   }
   public void onCreate(SQLiteDatabase db) {}
   public void onUpgrade(SQLiteDatabase database, int oldVersion, int
newVersion) {}
}
```

Advantages of SQLite

- SQLite can be used as a **temporary dataset** to do some processing on some data within an application.
- **Data storing** is very easy and efficient in SQLite. When you need to store an archive files SQLite can produce as smaller size archives and much amount of metadata included than Usual **ZIP archives**.
- SQLite is a very **light weight database**. So, it is a very good choice to develop embedded software for devices like televisions, cameras, Mobile phones, home electronic devices, etc.
- SQLite provides **an easy and efficient** way to deal with data rather than do the data processing internally inside in-memory variables.

Database helper class

```
public class DBHelper extends SQLiteOpenHelper {

   public static final String DATABASE_NAME = "MyDBName.db";
   public static final String CONTACTS_TABLE_NAME = "contacts";
   public static final String CONTACTS_COLUMN_ID = "id";
   public static final String CONTACTS_COLUMN_NAME = "name";
   public static final String CONTACTS_COLUMN_EMAIL = "email";
   public static final String CONTACTS_COLUMN_STREET = "street";
   public static final String CONTACTS_COLUMN_CITY = "place";
   public static final String CONTACTS_COLUMN_PHONE = "phone";
   private HashMap hp;

   public DBHelper(Context context) {
      super(context, DATABASE_NAME , null, 1);
   }
```

```
@Override
public void onCreate(SQLiteDatabase db) {
   // TODO Auto-generated method stub
   db.execSQL(
      "create table contacts " +
      "(id integer primary key, name text,phone text,email text, street text,place text)"
   );
}

@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
   // TODO Auto-generated method stub
   db.execSQL("DROP TABLE IF EXISTS contacts");
   onCreate(db);
}

public boolean insertContact (String name, String phone, String email, String street,String place) {
   SQLiteDatabase db = this.getWritableDatabase();
   ContentValues contentValues = new ContentValues();
   contentValues.put("name", name);
   contentValues.put("phone", phone);
   contentValues.put("email", email);
   contentValues.put("street", street);
   contentValues.put("place", place);
   db.insert("contacts", null, contentValues);
   return true;
}
```

```
public Cursor getData(int id) {
    SQLiteDatabase db = this.getReadableDatabase();
    Cursor res =  db.rawQuery( "select * from contacts where id="+id+"", null );
    return res;
  }

  public int numberOfRows(){
    SQLiteDatabase db = this.getReadableDatabase();
    int numRows = (int) DatabaseUtils.queryNumEntries(db, CONTACTS_TABLE_NAME);
    return numRows;
  }

  public boolean updateContact (Integer id, String name, String phone, String email, String street,String place) {
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues contentValues = new ContentValues();
    contentValues.put("name", name);
    contentValues.put("phone", phone);
    contentValues.put("email", email);
    contentValues.put("street", street);
    contentValues.put("place", place);
    db.update("contacts", contentValues, "id = ? ", new String[] { Integer.toString(id) } );
    return true;
  }
```

```
public Integer deleteContact (Integer id) {
      SQLiteDatabase db = this.getWritableDatabase();
      return db.delete("contacts",
      "id = ? ",
      new String[] { Integer.toString(id) });
   }

   public ArrayList<String> getAllCotacts() {
      ArrayList<String> array_list = new ArrayList<String>();

      //hp = new HashMap();
      SQLiteDatabase db = this.getReadableDatabase();
      Cursor res =  db.rawQuery( "select * from contacts", null );
      res.moveToFirst();

      while(res.isAfterLast() == false){
         array_list.add(res.getString(res.getColumnIndex(CONTACTS_COLUMN_NAME)));
         res.moveToNext();
      }
      return array_list;
   }
}
```

Sharing data between application using Content Providers

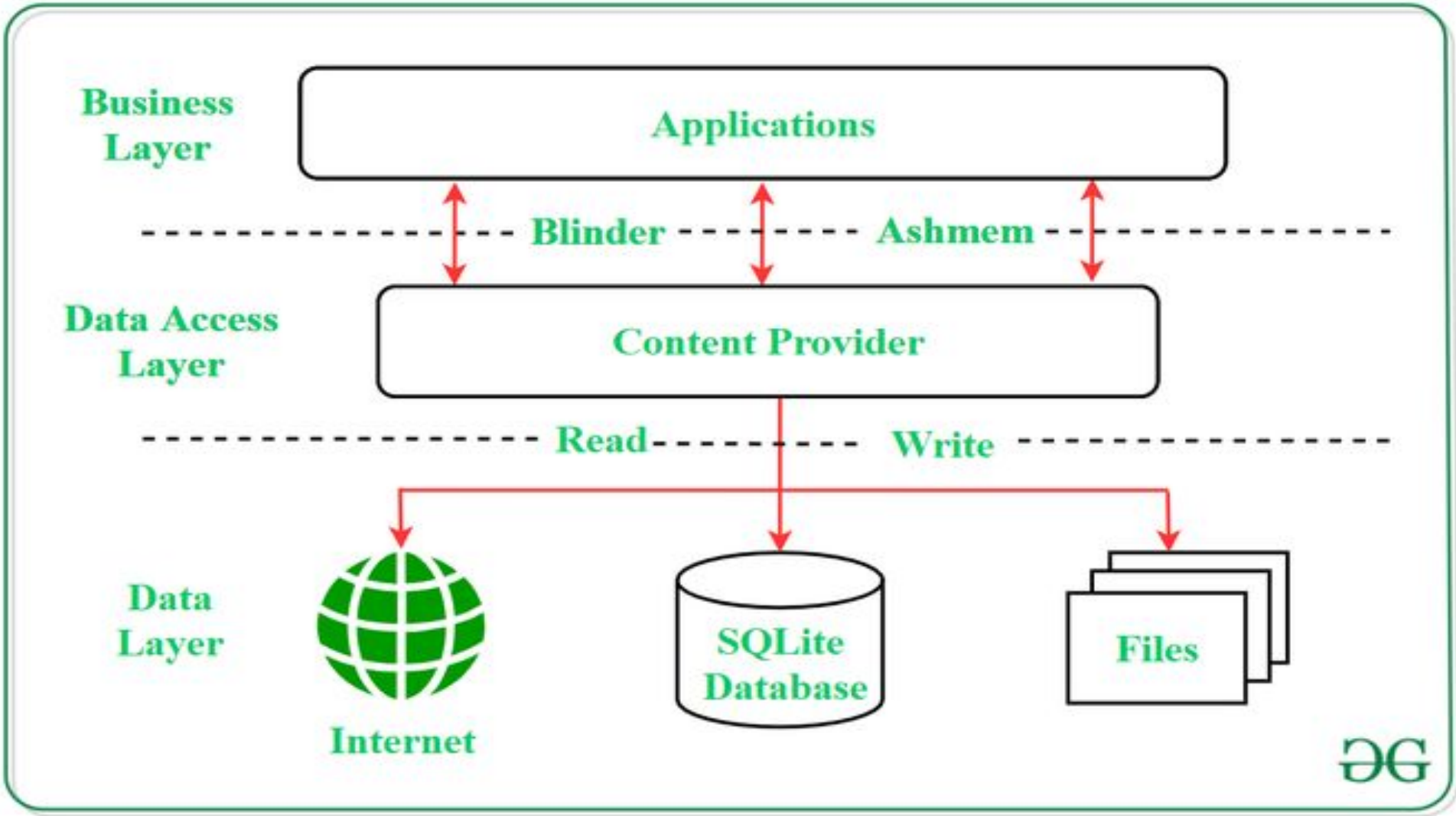
- ➔ On the Android platform, one application cannot directly access (read/write) other application's data.
- ➔ All persistent data of an app is private to that app.
- ➔ Every application has its own id data directory and own protected memory area.
- ➔ This means that an app cannot query or manipulate the data of another app.
- ➔ However, if you want to enable an app to query or manipulate the data of another app, you need to use the concept of content providers.

Content Provider:

Content provider is a set of data wrapped up in a custom API to read and write. It acts as an interface that allows you to store and retrieve data from a data source. And also allows you to share an app's data with other apps.

Content providers decouple the app layer from the data layer by abstracting the underlying data source, thereby making apps data-source independent. They allow full permission control by monitoring which app components or users should have access to the data making data sharing easy.

As a result, any app with appropriate permissions can add, remove, update, and retrieve data of another app including the data in some native Android databases.


Business Layer
Applications
Blinder
Ashmem
Data Access Layer
Content Provider
Read
Write
Data Layer
Internet
SQLite Database
Files

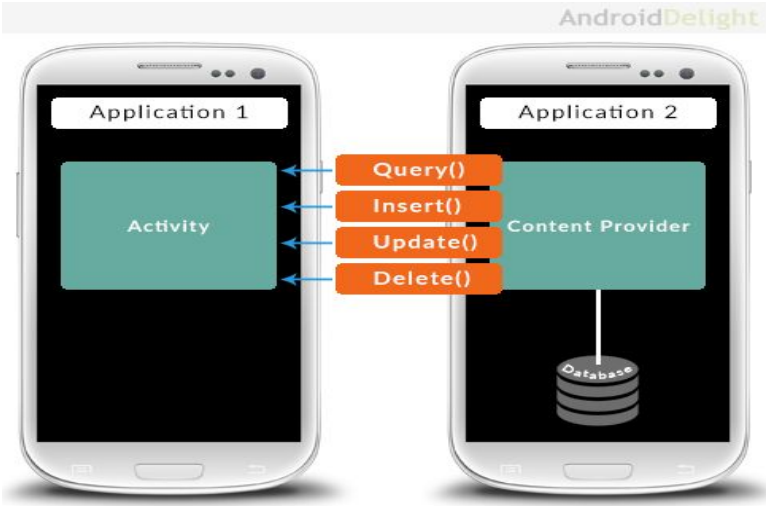
Types of content provider

There are 2 types of Content Providers :

- Custom content providers: These are created by the developer to suit the requirements of an app.
- Native content providers: These provide access to built-in databases, such as contact manager, media player, and other native databases. You need to grant the required permissions to your app before using native content providers.

Eg : The WhatsApp app sharing mobile contacts data.

Media store-Allows other applications to access, store media files.



Above diagram shows how content provider works. Application 2 stores its data in its own database and provides a provider. Application 1 communicates with the provider to access Application 2's data.

Creating a Content Provider :

Content Provider uses a special URI starting with ***content://*** will be assigned to each content providers and that will be recognized across applications.

To create a content provider we need to

1. Create sub class for ContentProvider.
2. Define content URI
3. Implement all the unimplemented methods. *insert()*, *update()*, *query()*, *delete()*, *getType()*.
4. Declare the content provider in AndroidManifest.xml

Specifying the URI of a Content Provider

Content provider URI consists of four parts.

content://authority/path/id

Each content provider exposes a URI that uniquely identifies the content provider's data set. If a content provider controls multiple datasets, a separate URI needs to be specified for each dataset. The URI of a content provider begins with the string, **content://**.

content:// - All the content provider URIs should start with this value

authority - A Java namespace of the content provider implementation. (fully qualified Java package name)

path - A virtual directory within the provider that identifies the kind of data being requested.

Id - Its optional part that specifies the primary key of a record being requested. We can omit this part to request all records.

Registering the provider in AndroidManifest.xml

As with any other major parts of Android application, we have to register the content providers too in the AndroidManifest.xml. ***<provider>*** element is used to register the content providers. It should be declared under ***<application>***.

```
<provider
    android:name=".MyProvider"
    android:authorities="com.example.contentproviderexample.MyProvider"
    android:exported="true"
    android:multiprocess="true" >
</provider>
```

Example: Sharing Contacts with App

1. Where are contacts stored?

- Android stores your **contacts** in a system-wide database.
- This database is exposed through the **Contacts Content Provider**.
- URI for contacts:

  ContactsContract.Contacts.CONTENT_URI
- Any app (WhatsApp, Truecaller, Gmail, etc.) can read contacts from this provider **if the user grants permission**.

2. App Accessing Contacts

When you install App, it asks for this permission:

<uses-permission android:name="android.permission.READ_CONTACTS"/>

3. Querying Contacts (like App does)

Here's how an app can fetch all contacts from the **Contacts Content Provider**:

```
Cursor cursor = getContentResolver().query(
        ContactsContract.Contacts.CONTENT_URI,
        null,   // which columns
        null,   // WHERE clause
        null,   // selectionArgs
        null    // sort order
);

if (cursor != null && cursor.moveToFirst()) {
    do {
        String contactID = cursor.getString(
                cursor.getColumnIndexOrThrow(ContactsContract.Contacts._ID));
        String displayName = cursor.getString(
                cursor.getColumnIndexOrThrow(ContactsContract.Contacts.DISPLAY_NAME));
        Log.d("CONTACT", "ID: " + contactID + " Name: " + displayName);

    } while (cursor.moveToNext());
    cursor.close();
}
```

Json Parsing

JSON (JavaScript Object Notation) is a straightforward data exchange format to interchange the server's data, and it is a better alternative for XML. This is because JSON is a lightweight and structured language.

Android supports all the JSON classes such as **JSONStringer, JSONObject, JSONArray,** and all other forms to parse the JSON data and fetch the required information by the program.

JSON's main advantage is that it is a language-independent, and the JSON object will contain data like a key/value pair.

JSON nodes will start with a **square bracket ([)** or with a **curly bracket ({)**.

The square and curly bracket's primary difference is that the **square bracket ([)** represents the beginning of a **JSONArray node**.

The **curly bracket ({)** represents a **JSONObject**.

So one needs to call the appropriate method to get the data.

Sometimes JSON data start with [. We then need to use the **getJSONArray()** method to get the data.

Similarly, if it starts with {, then we need to use the **getJSONobject()** method.

The syntax of the JSON file is as following:

```
{ "Employee" :
    [
     {"id":"101","name":"Sonoo Jaiswal","salary":"50000"},
     {"id":"102","name":"Vimal Jaiswal","salary":"60000"}
    ]
}
```

Types of JSON

1. **JSON Object** – { key : value }
   Example:

   { "id": 1, "name": "Preety" }

2. **JSON Array** – [{ ... }, { ... }]

   Example:

   [

   { "id": 1, "name": "Preety" },

   { "id": 2, "name": "Tilwani" }

   ]

JSON Parsing in Android

Android provides different ways to parse JSON:

1. Using JSONObject & JSONArray (Native JSON classes)

- Best for simple JSON data.
- Available in org.json package.

{ "id": 101, "name": "Preety", “email”: “preety@example.com” }

Example:

```
import org.json.JSONArray;
import org.json.JSONException;
import org.json.JSONObject;

public class JSONParsingDemo {
    public static void main(String[] args) {
        String jsonString = "{ \"id\":101, \"name\":\"Preety\", \"email\":\"preety@example.com\" }";
        try {
            // Convert string to JSONObject
            JSONObject obj = new JSONObject(jsonString);
            int id = obj.getInt("id");
            String name = obj.getString("name");
            String email = obj.getString("email");
            System.out.println("ID: " + id);
            System.out.println("Name: " + name);
            System.out.println("Email: " + email);
        } catch (JSONException e) {
            e.printStackTrace();
        }
    }
}
```

2. Parsing JSON Array

```
String jsonArrayString = "[{\"id\":1,\"name\":\"Preety\"},{\"id\":2,\"name\":\"Tilwani\"}]";

try {
    JSONArray jsonArray = new JSONArray(jsonArrayString);

    for (int i = 0; i < jsonArray.length(); i++) {
        JSONObject obj = jsonArray.getJSONObject(i);
        int id = obj.getInt("id");
        String name = obj.getString("name");

        System.out.println("ID: " + id + ", Name: " + name);
    }

} catch (JSONException e) {
    e.printStackTrace();
}
```

3. Using Gson (Google Library)

- Gson makes parsing easier by **mapping JSON directly to Java objects**.
- Add dependency in build.gradle:

implementation 'com.google.code.gson:gson:2.10.1'

Example:

```
import com.google.gson.Gson;

class User {
    int id;
    String name;
    String email;
}
public class GsonDemo {
    public static void main(String[] args) {
        String jsonString = "{ \"id\":101, \"name\":\"Preety\", \"email\":\"preety@gmail.com\" }";
        Gson gson = new Gson();
        User user = gson.fromJson(jsonString, User.class);
        System.out.println("ID: " + user.id);
        System.out.println("Name: " + user.name);
        System.out.println("Email: " + user.email);
    }
}
```

For **arrays with Gson**:

```
User[] users = gson.fromJson(jsonArrayString, User[].class);

for (User u : users) {

    System.out.println(u.id + " - " + u.name);

}
```

4. Using Retrofit (with Gson or Moshi)

- For **real-world APIs**, Retrofit is commonly used.
- Retrofit handles networking + parsing JSON automatically into Java objects.

Example:

```
String url = "https://jsonplaceholder.typicode.com/users";
RequestQueue queue = Volley.newRequestQueue(this);
JsonArrayRequest request = new JsonArrayRequest(Request.Method.GET, url, null,
        response -> {
            try {
                for (int i = 0; i < response.length(); i++) {
                    JSONObject user = response.getJSONObject(i);
                    String name = user.getString("name");
                    String email = user.getString("email");

                    Log.d("JSON", "User: " + name + ", Email: " + email);
                }
            } catch (JSONException e) {
                e.printStackTrace();
            }
        },
        error -> Log.e("JSON", "Error: " + error.getMessage()));
queue.add(request);
```

Services

An Android **Service** is a fundamental application component used to perform long-running operations in the background without a user interface. Services continue to run even if the user switches to another application.

Here is a comprehensive breakdown of Android Services:

What are Services used for?

- **Long-Running Background Tasks:** Operations that need to continue without interruption, such as playing music, handling network transactions, or downloading a large file.
- **Interprocess Communication (IPC):** Allowing application components (like an Activity) or even other applications to bind to the service to interact with it and send requests.
- **Running Code Independent of the UI:** Since a service has no user interface, it is not tied to the lifecycle of an Activity.

Types of Android Services

Android services are generally categorized based on how they run and interact with the user:

Foreground Service:

Performs an operation that is **noticeable to the user**. It must display a **notification** in the status bar while running. This gives the task a higher priority, making it less likely for the system to kill the service for low memory.

For example:

A **music player** that continues to play music after the user leaves the app, showing a "Now Playing" notification.

```
startForegroundService(new Intent(this, MusicService.class));
```

Background Service:

Performs an operation that is **not directly noticed by the user**. These services are subject to significant system constraints, especially on modern Android versions (Android 8.0+), and are often killed to reclaim resources. For simple, deferred background work, **WorkManager** is the recommended alternative.

For Example:

Periodically **syncing data** to a server that doesn't need a constant update notification.

Bound Service:

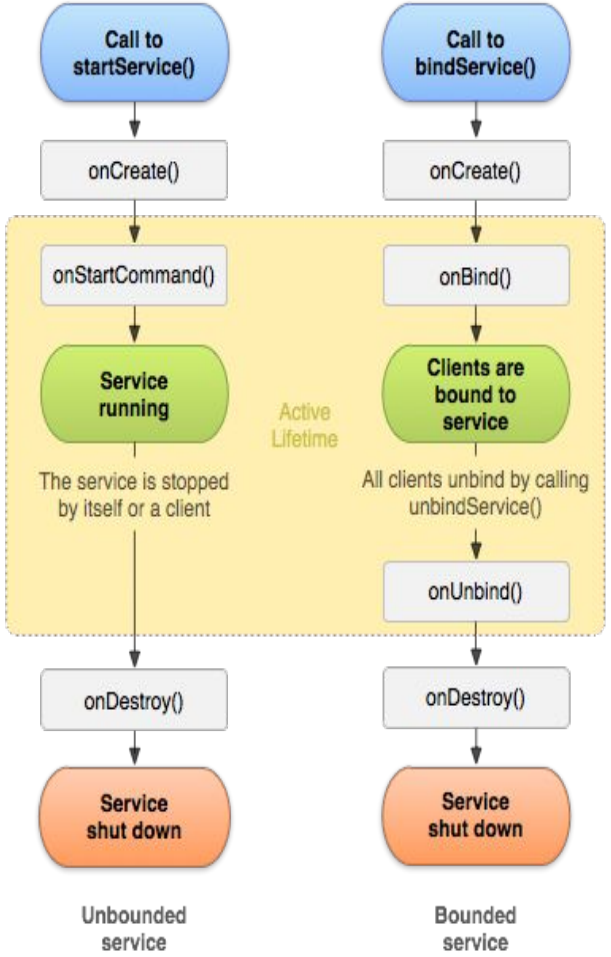
Provides a **client-server interface** that allows application components (clients) to bind to the service, send requests, receive results, and even perform IPC. A bound service runs only as long as another application component is bound to it.

For Example:

A service that offers a utility function to other components, such as a **spell checker** service.

```
bindService(intent, serviceConnection, BIND_AUTO_CREATE);
```

Service Lifecycle



```
public class ExampleService extends Service {
    int startMode;       // indicates how to behave if the service is killed
    IBinder binder;      // interface for clients that bind
    boolean allowRebind; // indicates whether onRebind should be used

    @Override
    public void onCreate() {
        // The service is being created
    }
    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        // The service is starting, due to a call to startService()
        return startMode;
    }
    @Override
    public IBinder onBind(Intent intent) {
        // A client is binding to the service with bindService()
        return binder;
    }
    @Override
    public boolean onUnbind(Intent intent) {
        // All clients have unbound with unbindService()
        return allowRebind;
    }
    @Override
    public void onRebind(Intent intent) {
        // A client is binding to the service with bindService(),
        // after onUnbind() has already been called
    }
    @Override
    public void onDestroy() {
        // The service is no longer used and is being destroyed
    }
     }
```

Service Lifecycle Methods

`onCreate()`: When the service is first created.One-time setup tasks, like initialising variables. This is called only once for the service's entire life.

`onStartCommand()`: When a component calls `startService()`.The entry point for a **started** service. Contains the code to execute the desired background task. If the system kills the service and resources become available, the return value dictates how the system should handle restarting it (e.g., `START_STICKY`).

`onBind()`: When another component calls `bindService()`. **Mandatory to implement.** If the service is a **bound** service, this method must return an `IBinder` object for the client to interact with the service. If the service is not meant to be bound, it should return `null`.

`onUnbind()`: When all clients have disconnected from a bound service.Clean up resources before the service is destroyed. If this method returns `true`, the service will receive a call to `onRebind()` if a new client binds to it later.

`onDestroy()`: The final call before the service is removed from memory.Clean up all remaining resources, stop any running threads, and unregister listeners.

Alternatives for Background Work:

WorkManager:

Guaranteed execution for deferred but necessary background work (e.g., data sync, sending logs), especially when constraints (like network or battery status) are important.

JobScheduler:

Scheduling individual, system-managed background jobs (now largely superseded by WorkManager).

Threads/Coroutines: Short-duration, non-critical background work that needs to be performed only while the application's activity is in the foreground.